

ECE456 Network Centric Programming – System Calls and Threads

1. (6 points) What is the output?

```
1 #include <fcntl.h>
2 #include <unistd.h>
3
4 int main() {
5
6     int fd = open("test.txt", O_WRONLY | O_CREAT, 0644);
7
8     write(fd, "ABC", 3);
9     write(fd, "DEF", 3);
10
11     close(fd);
12 }
```

Assume the file did not previously exist.

What are the contents of `test.txt` after the program runs?

2. (6 points) Fill in the blank so the program reads 4 bytes from a file.

```
1 #include <fcntl.h>
2 #include <unistd.h>
3
4 int main() {
5
6     int fd = open("data.bin", O_RDONLY);
7
8     char buf[4];
9
10    _____(fd, buf, 4);
11
12    close(fd);
13 }
```

3. (6 points) What error occurs? (single word)

```
1 #include <fcntl.h>
2 #include <unistd.h>
3
4 int main() {
5
6     int fd = open("file.txt", O_RDONLY);
7
8     close(fd);
9
10    char buf[10];
11    read(fd, buf, 10);
12 }
```

Error:

4. (6 points) What system call creates a directory?

```
1 #include <sys/stat.h>
2
3 int main() {
4
5     -----("newdir", 0755);
6 }
```

5. (6 points) What does the following program change?

```
1 #include <unistd.h>
2
3 int main() {
4
5     chdir("logs");
6 }
```

Short answer:

6. (10 points) Consider the following program.

```
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main() {
5
6     printf("A\n");
7
8     fork();
9
10    printf("B\n");
11 }
```

List two possible outputs.

7. (10 points) Consider:

```
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main() {
5
6     printf("1\n");
7
8     pid_t pid = fork();
9
10    if(pid == 0)
```

```

11     printf("child\n");
12 else
13     printf("parent\n");
14 }

```

How many processes run the program after the fork?

Answer:

List two possible outputs.

8. (12 points) Suppose a program writes:

```

1 fprintf(stdout, "hello\n");
2 fprintf(stderr, "error\n");

```

The program is executed as:

```
./prog > out.txt 2> err.txt
```

Where does each line go?

```

hello  -----
error  -----

```

9. (12 points) Explain what the following command does.

```
./prog > output.txt 2>&1
```

Short answer:

10. (15 points) Write code that:

- opens a file called `numbers.txt`
- writes the numbers 0–9 (one per line)
- closes the file

Use only:

- `open`
- `write`
- `close`